B.Sc. in Computer Science and Engineering Thesis

Towards Practical On-Device Android Malware Detection via Lightweight Static Analysis

Submitted by

Abdullah Faiyaz
2005065

Jarin Tasnim
2005083

Mohammad Sadnam Faiyaz
2005111

Supervised by

Dr. Md. Mostofa Akbar

Department of Computer Science and Engineering
Bangladesh University of Engineering and Technology

Dhaka, Bangladesh

June, 2026

CANDIDATES' DECLARATION

This is to certify that the work presented in this thesis, titled, "Towards Practical On-Device Android Malware Detection via Lightweight Static Analysis", is the outcome of the investigation and research carried out by us under the supervision of Dr. Md. Mostofa Akbar.

It is also declared that neither this thesis nor any part thereof has been submitted anywhere else for the award of any degree, diploma or other qualifications.

Abdullah Faiyaz
2005065

Jarin Tasnim
2005083

Mohammad Sadnam Faiyaz
2005111

ACKNOWLEDGEMENT

We are thankful to

Finally,

Dhaka
June, 2026

Abdullah Faiyaz

Jarin Tasnim

Mohammad Sadnam Faiyaz

Contents

List of Figures

List of Tables

List of Algorithms

ABSTRACT

Detection of malware on Android has come a long way, with both static and dynamic analysis methods. But the question of implementing them in real-life, constrained environments with limited battery, CPU, and memory still remains answered by very few approaches. Taking the question seriously, we present VigiDroid, a fully offline static analysis framework that runs entirely on the device itself. We have experimented with different feature sets (manifest attributes, API calls, DEX header metadata, and raw APK byte sequences), pairing each with lightweight classifiers such as XGBoost, multilayer perceptrons, and 1D CNNs. Every model is exported to ONNX and executed on-device via ONNX Runtime, enabling direct evaluation on Android devices. In addition to accuracy and F1-score, we profile feature extraction, vectorization, and inference times, CPU utilization, and memory consumption. Our results suggest that lightweight static features extracted from DEX headers, permissions, intents, and broadcast receivers offer one of the most favorable accuracy-resource trade-offs (96.99% accuracy and F1 of 0.9503 with only 0.9 ms CPUtime and 0.5 ms walltime).

Chapter 1

Introduction

The rapid proliferation of smartphones has fundamentally reshaped human communication, commerce, and daily productivity. At the center of this mobile revolution is the Android operating system, which commands a dominant global market share powering billions of active devices. Because of its open-source nature and massive user base, Android has also become the primary target for cybercriminals, resulting in an exponential rise in sophisticated mobile malware. Traditional signature-based antivirus solutions struggle to defend against zero-day exploits and obfuscated payloads, forcing modern research toward machine learning-based detection models.

However, a major gap persists between theoretical malware detection and practical deployment. Most state-of-the-art frameworks rely on resource-heavy dynamic analysis or complex structural decompilation that must be offloaded to cloud servers. This dependency poses significant user privacy risks, demands continuous network connectivity, and inflicts heavy battery and memory penalties if executed locally.

To address these limitations, this thesis introduces VigiDroid: a fully offline, multi-tiered static analysis framework designed to run entirely on resource-constrained mobile hardware. By systematically exploiting lightweight metadata—starting with the structural blueprints found inside an application's compressed archive—VigiDroid seeks to prove that high-fidelity security enforcement can be achieved locally, efficiently, and without compromising device longevity.

1.1 Android Operating System and Ecosystem

The mobile computing landscape is heavily dominated by the Android operating system, an open-source platform maintained by Google and built upon a modified version of the Linux kernel. Because of its open-source nature, flexibility, and extensive developer ecosystem, Android powers billions of active devices worldwide, ranging from smartphones and tablets to

wearables and smart televisions.

Unlike desktop operating systems, Android isolates applications from one another using a security mechanism known as application sandboxing. Every Android application runs inside its own low-privileged process and is assigned a unique Linux User ID (UID). This ensures that, under normal operational boundaries, an application cannot access the private data, memory, or execution space of another application without explicit permission.

1.2 The Architecture of an Android Package (APK)

To deploy software within this ecosystem, applications must be compiled and bundled into a single file format known as an Android Package (APK), which carries the .apk file extension. Structurally, an APK file is not a solid binary executable; rather, it is a standard ZIP archive compressed according to specific structural standards. When an APK file is decompressed or inspected, it reveals a standardized directory layout containing all components necessary for the application's runtime execution:

- classes.dex: The compiled Dalvik Executable file containing the executable bytecode that runs on the Android Runtime (ART) or legacy Dalvik Virtual Machine. This file contains the compiled application logic and API references.
- res/ and assets/: Directories storing compiled and raw application resources, such as user interface layouts, image assets, localized strings, and raw binary attachments.
- META-INF/: A folder storing the cryptographic signatures and certificates of the application, ensuring tamper-detection and origin verification during app updates or installation.
- AndroidManifest.xml: The central structural and architectural blueprint of the application.

1.3 The Lifecycle and Execution of an APK

When a user installs or executes an application, the Android system interacts with the APK as a structural package:

- Parsing and Verification: The system's Package Installer extracts and reads the cryptographic structures inside META-INF/ to verify authenticity. Concurrently, the platform parses structural definitions to register the application with the system services.

- Compilation and Optimization: Upon installation or during idle device states, the Android Runtime (ART) compiles the Dalvik bytecode contained within classes.dex into native machine code using Ahead-Of-Time (AOT) or Just-In-Time (JIT) compilation.
- Process Sandboxing: When the user taps the app icon, the system forks a secure process via the Zygote base process, assigns its distinct UID, maps its sandboxed storage directories, and initiates execution.

1.4 The Role and Importance of AndroidManifest.xml

The AndroidManifest.xml file is the most critical metadata component within the APK archive. It serves as an explicit declaration contract between the application and the Android operating system kernel. The system cannot launch any application component or grant any hardware access if it is not formally declared inside this manifest file.

However, inside a compiled APK, the manifest file is not saved as human-readable clear text. To reduce package size and optimize on-device parsing speeds, the compilation toolchain processes the text-based XML file into a compressed binary format known as Binary XML (AXML). This binary format substitutes strings with global string pool offsets and organizes structural nodes into tokens.

The manifest file defines the complete configuration boundary of an application by specifying:

- Package Identity: The unique Java-style package namespace identifier used by the operating system to track the app.
- Application Components: The declaration of the four fundamental building blocks of Android:
  - Activities: The visible user interface screens.
  - Services: The long-running background operations that lack a user interface.
  - Broadcast Receivers: Event listeners that allow the app to receive system-wide broadcast intents (e.g., detecting if the charger is connected or if the device finished booting).
  - Content Providers: Interfaces for sharing structured relational data between sandboxed applications.
- Intent Filters: Declarations that specify what hardware or software events an app component can respond to.

- Permission Declarations: The security boundaries of the app. It declares Requested Permissions (¡usespermission¿), which specify the sensitive resources the app needs to access (e.g., READ_SMS, CAMERA, ACCESS_FINE_LOCATION), and Defined Permissions, which restrict access to the app's own components.

5. Manifest-Driven Analysis in VigiDroid Within our proposed framework, VigiDroid, the AndroidManifest.xml serves as the primary, high-velocity defensive gating layer during on-device scanning.

1.5 Why We Check AndroidManifest.xml

Malicious applications routinely reveal their behavioral intents and structural anomalies within their manifest metadata prior to running any bytecode. For instance, a benign calculator application has no operational justification to request the SEND$_SMS, RECEIVE_BOOT_COMPLETED, andR$

Because extracting and parsing the full compiled bytecode (classes.dex) or disassembling structural call graphs is heavily limited by the CPU, battery, and memory constraints of a mobile device, VigiDroid leverages the manifest file as a lightweight, first-line statistical filter. Reading the binary manifest requires minimal file decompression and I/O overhead compared to processing the entire APK, making it ideal for immediate, on-device triaging.

1.6 VigiDroid Does With It

When a scan is initiated on an uninstalled or target APK file, VigiDroid performs the following sequential actions:

- Targeted Streaming Access: Instead of unzipping the complete APK into a temporary folder, VigiDroid opens an optimized stream to isolate and extract only the binary AndroidManifest.xml file directly from the compressed ZIP archive.
- Binary XML Decoding: Our Java-backed processing pipeline parses the compressed binary AXML layout into an internal structural representation without needing external third-party decompilation utilities.
- Feature Extraction and Tokenization: The parsing engine extracts three explicit feature dimensions:
  - Permissions: The full string array of requested permissions.

  - Intent Filters: The implicit actions and structural categories declared.
  - Broadcast Receivers: Statically registered event receivers.

- Vocabulary Normalization: These text strings are checked against an internal token vocabulary dictionary pre-compiled within the app assets, transforming varied string values into an optimized, fixed-width binary vector representation (1 for present, 0 for absent).
- Multi-Tier Execution: This vectorized manifest representation is fed directly into our initial scanning tiers (Tier 1 and Tier 2) using optimized machine learning classifiers running via ONNX Runtime. If these lightweight tiers return a high-confidence prediction, VigiDroid completes the scan immediately, saving valuable battery and CPU cycles by skipping deep structural bytecode analysis entirely.

1.7 Thesis Contributions and Outline

The primary contributions of this research are threefold:

- The architecture and end-to-end implementation of VigiDroid, a native-backed, on-device Android scanning engine optimized via ONNX Runtime.
- A rigorous empirical evaluation of multiple feature-model pairings (ranging from permission vectors to raw byte sequences) across temporal data splits to isolate the optimal accuracy-resource frontier.
- The design of an intelligent 4-tier cascading scan orchestration system that minimizes hardware overhead by bypassing heavy bytecode inspection whenever lightweight manifest tokens provide high-confidence triage.

The remainder of this thesis book is organized as follows:

- Chapter 2: Background and Related Work provides a formal introduction to the Android compilation ecosystem and surveys previous static, dynamic, and hybrid machine learning approaches, highlighting their resource trade-offs.
- Chapter 3: Methodology and System Design details VigiDroid's underlying architecture, from targeted archive parsing and feature tokenization to the multi-tier cascading policy engine.
- Chapter 4: Experimental Setup and Implementation describes the datasets used, training paradigms, ONNX optimization steps, and the specialized hardware-profiling harness deployed on physical test devices.

- Chapter 5: Results and Discussion presents a comparative analysis of the model configurations, benchmarking detection accuracy and F1-scores directly against CPU time, memory footprint, and battery drain.
- Chapter 6: Conclusion and Future Work summarizes the core insights of this study and outlines future optimization paths for on-device mobile security frameworks.

Chapter 2

Background and Related Works

Android malware detection has attracted significant research attention over the past decade. Researchers have proposed a wide spectrum of techniques—ranging from lightweight static analysis to heavyweight dynamic execution monitoring—each offering different trade-offs between detection accuracy, computational cost, and deployment practicality. This section surveys the key prior works that directly inform VigiDroid's design philosophy, feature engineering choices, model architecture, and on-device deployment strategy.

2.1 LinRegDroid: Permission-Based Detection via Multiple Linear Regressionv

Proposed by Narayanan et al., [1] represents one of the early systematic efforts to apply statistical machine learning to Android malware detection using exclusively permission metadata. The central motivation behind this work is that Android permissions requested by an application at install time reveal its underlying behavioral intent. Malicious applications tend to request permissions that are unusual in combination or overly broad relative to their stated utility. LinRegDroid capitalizes on this insight by parsing the AndroidManifest.xml of each APK to extract the requested permissions, encoding them as binary feature flags (1 if requested, 0 if not), and training multiple linear regression classifiers to distinguish between benign and malicious samples. The authors demonstrated that this method is computationally inexpensive, requiring no bytecode extraction or dynamic execution, which makes it ideal for constrained environments.

This work directly motivates VigiDroid's exploration of permission-only feature representations as a baseline configuration. The simplicity of permission flags aligns perfectly with VigiDroid core objective: minimizing feature extraction overhead during on-device scanning. VigiDroid acknowledges that while permission signals lack deep expressive power compared to bytecode

structural features, they provide a compelling accuracy-to-cost trade-off that makes them viable in a lightweight scanning tier.
VigiDroid implements a dedicated LinRegDroid-inspired model configuration within its experimental pipeline. Specifically, the framework includes a logistic regression baseline (evaluated as LinRegDroid in the thesis metrics) that uses manifest permission flags as the sole feature input. The manifest parser in VigiDroid's Java-based feature extraction layer produces binary permission flag vectors compatible with this model. In on-device profiling, this implementation achieves an accuracy of 91.75% and an F1-score of 0.8645 with a sub-millisecond inference latency (0.7 ms), establishing an important lower-bound resource reference point against which richer feature configurations are compared.

2.2 Detecting Android Malware by Mining Statically Registered Broadcast Receivers

Mohsen and Ismail presented [2] static analysis approach centered on broadcast receivers as the primary discriminating feature for malware detection. Broadcast receivers are core components that listen for system-wide events (e.g., device boot completion, network status changes, incoming SMS). The authors argue that malicious applications disproportionately register broadcast receivers corresponding to sensitive system events, such as BOOT_COMPLETED (to achieve persistence after a reboot) or SMS_RECEIVED (to intercept credentials). Their methodology mines statically declared broadcast receiver actions from the manifest, constructs a presence-absence feature representation, and trains a machine learning classifier. This signal is highly efficient to extract and orthogonal to pure permission-based approaches.

This work informs VigiDroid's inclusion of broadcast receiver actions as an independent feature modality. The insight that receiver patterns carry independent discriminative signals beyond permissions motivates VigiDroid's feature fusion experiments.

VigiDroid's Java pipeline features a dedicated broadcast receiver parser that enumerates all ¡receiver¿ elements in the parsed manifest and collects their ¡intent-filter¿ action strings. More significantly, this work is synthesized into our best-performing model configuration: Dex+Broadcast Fusion. By combining DEX header metadata with broadcast receiver features, this configuration achieves 96.99% accuracy and an F1-score of 0.9503 at an exceptionally low CPU time of only 0.9 ms, demonstrating that combining structural bytecode signals with receiver action patterns yields an optimal accuracy-resource frontier.

2.3 One-Dimensional Convolutional Neural Networks For Android Malware Detection

Hasegawa and Iyatomi [3] shifted away from structured manifest files to evaluate domain-agnostic, raw byte signals. Their approach trains 1-D Convolutional Neural Networks (CNNs) directly on the raw bytes of an unextracted APK file, testing various sample lengths ($N = 512, 1024, 2048, 4096$) from both the beginning and the end of the file. Crucially, they discovered that analyzing the last N bytes (the tail) of the APK zip archive provides significantly higher classification performance than the first N bytes. Because APK files are fundamentally ZIP archives, the final segments house critical payload structures, directory indices, compiled assets, and occasionally malicious payloads appended to evade typical signature boundaries.

This work serves as the foundational reference for VigiDroids deep learning evaluation tier. It motivates our framework's capacity to bypass structural extraction altogether when a comprehensive file inspection is requested, evaluating whether raw signal processing can out-compete domain-expert feature extraction under rigorous on-device resource constraints.
We implemented a self-contained 1-D CNN model that replicates this paradigm. VigiDroid's feature extraction layer includes a tail-byte reader that slices exactly the last 1024 bytes of the raw APK file without extracting its contents. This array is forwarded into a 1-D CNN exported to ONNX. In our multi-tier deployment, this model operates as our Tier 4 heavy-fusion fallback checkpoint, balancing raw sequence-level accuracy against the specialized structural patterns found in earlier tiers.

2.4 SeqMobile: An Efficient Sequence-Based Malware Detection System Using RNN on Mobile Devices

Feng et al. [4] proposed SeqMobile, a pioneering framework for feasible, fully on-device malware detection using Recurrent Neural Networks (RNNs). They extracted both non-sequential features (permissions, intent filters) and sequential features (API call sequences, intent sequences) from the AndroidManifest.xml and the classes.dex binary. To run efficiently on-device, they built specialized feature dictionaries (mapping tokens to unique integers) and implemented native code to extract semantic sequences rapidly. They also optimized the architecture by removing consecutive repetitive calls to reduce sequence sizes by approximately 57% with minimal accuracy degradation. The trained models were deployed on-device using TensorFlow Lite, comparing dynamic non-quantized and static quantized models.
SeqMobile establishes the performance ceiling for on-device sequence processing and directly

validates that mobile hardware can handle deep neural architectures if features are properly vectorized. However, SeqMobile's reliance on full API call graph sequencing entails heavy extraction penalties and noticeable runtime delays (2.9s per prediction sequence on a Samsung S10+). This inspired VigiDroid to look for structural shortcuts—such as DEX headers—to capture bytecode characteristics without parsing complete control flow graphs.
While VigiDroid replaces heavy sequential RNNs with highly optimized structures like XGBoost and 1-D CNNs to avoid multi-second scanning delays, it adopts SeqMobile's core design choice: performing all feature parsing, vocabulary normalization, and tensor vectorization entirely locally within native-backed components. Furthermore, we utilize their vocabulary dictionary mapping methodology to parse permissions and intent strings into structural indices optimized for ONNX Runtime consumption.

2.5 A Lightweight Multi-Source Fast Android Malware Detection Model

This work [5] a multi-source fusion methodology utilizing four distinct lightweight base models to perform ensemble learning.

- Base Model 1 (MLP-H): Extracts and parses structural header bytes from the classes.dex file and passes them to a Multilayer Perceptron.
- Base Model 2 (MLP-P): Computes the mathematical entropy of the classes.dex file and extracts power spectral density features.
- Base Model 3 (ASCNN-I): Extracts manifest permissions/intents using a bag-of-words model, shrinking the dimensions via an Adaptive Shrinkage Convolutional Neural Network (ASCNN).
- Base Model 4 (ASCNN-C): Concatenates the DEX header features with manifest bag-of-words features through a dual-branch ASCNN architecture. The predictions of these models are fused via an adaptive soft voting ensemble policy and tested live on Android devices.

This work serves as the architectural bedrock for VigiDroid's multi-source feature extraction. It highlights that analyzing the structural file metadata of the DEX header provides deep bytecode insights at a fraction of the cost of full disassembling or decompiling. VigiDroid expands on this by introducing temporal evaluation splits and systematically profiling execution resource bounds (battery, memory, CPU time).

We fully implemented several model variations inspired by this multi-source paradigm. VigiDroid includes a standalone MLP-H configuration that parses the raw classes.dex headers directly from the APK archive. We also implemented the dual-branch feature architecture (Dex+Manifest ASCNN and Dex+Manifest Dual). Most importantly, VigiDroid leverages this work to build a 4-tier cascading scan orchestration system, sequentially deploying these feature extractions in order of computational weight.

2.6 SAMADroid: A Novel 3-Level Hybrid Malware Detection Model

Arshad et al. proposed [6] SAMADroid, a hybrid, three-level detection framework designed to combine static and dynamic analysis. The model splits its workload between local and remote environments: lightweight static analysis (permissions, structural manifest components) runs locally on-device to minimize performance overhead, while resource-intensive dynamic analysis (monitoring real-time system calls and network activity) is offloaded to remote host servers.

SAMADroid frames the critical trade-off between local processing limits and multi-level enforcement policies. However, its architecture exhibits two clear limitations in modern environments. First, modern Android security models (Android 10 and above) strictly prevent sandboxed applications from launching or tracing other active applications' behaviors, rendering its dynamic capture client obsolete without root privileges. Second, its dependency on active network offloading compromises user privacy and breaks down in offline scenarios. This directly motivated VigiDroid to pursue a fully offline, 100% on-device cascading pipeline that achieves hybrid-level accuracy without offloading data or violating platform sandbox boundaries.

While VigiDroid completely discards the remote dynamic offloading component due to security and privacy boundaries, it refines SAMADroid's multi-level gating concept. Instead of a local-to-remote split, VigiDroid implements this multi-level concept entirely on-device via a four-tier cascading policy. The initial tiers execute lightweight static checks, and only move to deeper, more complex structural extraction (such as raw byte CNNs) when intermediate classification margins are uncertain.

2.7 DL-Droid: Deep Learning Based Android Malware Detection Using Real Devices

Alzaylaee et al. [7]created DL-Droid, a framework dedicated to executing deep learning models driven by dynamic features extracted directly from within real physical devices. Their work proved that deep learning classifiers trained on dynamic features (like runtime execution logs and state transitions) can reach exceptionally high performance ceilings and capture zero-day obfuscated malware strains that slip past static signature frameworks.

DL-Droid provides a standard for performance accuracy, but highlights the significant real-world costs of dynamic analysis. Requiring every app to be completely installed and executed for an extended period to catch background hooks is highly impractical for daily on-demand device scanning, consuming immense battery and time. This limitation reinforces VigiDroid's core hypothesis: lightweight, multi-tiered static structural models can achieve a highly favorable accuracy-resource frontier entirely locally, without requiring app installation or full execution.

VigiDroid uses the performance metrics of DL-Droid as an upper-bound comparison baseline in its thesis evaluation suite. This allows us to explicitly demonstrate how closely VigiDroid's sub-millisecond, on-device cascading static scan modes approach the detection accuracy of comprehensive dynamic frameworks, while completely avoiding execution overhead and device installation requirements.

Chapter 3

Proposed Methodology

3.1 Model Development

The offline half of our pipeline follows a consistent eight-phase structure (P0–P8) applied independently to each detector: environment setup and configuration, APK corpus indexing, feature extraction, dataset loader construction, model definition, training, held-out test evaluation, ONNX export, and finally a parity check confirming that the exported model reproduces PyTorch probabilities within 10^{-4} on a set of representative samples. Enforcing this parity gate before any Android integration ensures that accuracy figures reported from the PC are meaningful on-device.

Our training corpus comprises roughly 500 GB of APKs drawn from AndroZoo and related sources, organized by year and label. We adopt a temporal split throughout: models are trained on 2020–2021 samples and evaluated on 2022–2023 samples, which mirrors the realistic scenario where a deployed detector faces APKs newer than those it was trained on. Validation folds are always drawn from the training years to prevent future leakage.

We implement nine model families spanning a range of feature domains and computational costs, grouped into three inference tiers. At the **LIGHT** tier, four fast models handle manifest-level and raw-byte signals: **ByteCNN** [3] applies a 1-D convolutional network to the last 1024 bytes of the APK; **Linregdroid** [1] feeds a normalized manifest permission vector into a linear classifier; **MLDP-pruned** [2] uses a compact MLP over a permission subset selected by the MLDP framework; and a **Broadcast+MLDP hybrid** [**?**, 2] combines static receiver-action features with MLDP permissions through a small fusion network. At the **MID** tier, we add structural DEX signals: **MLP(H)** (BM1), a 104-dimensional normalized DEX header classifier inspired by MSFDroid; **Pattern A**, which performs early fusion of the DEX header with a manifest bag-of-words over a roughly 4,380-entry lexicon; **Pattern B**, a dual-branch variant that fuses the same two signals via late fusion; and two **MLDP+DEX header cascade** configura-

tions (modes A and B) that route samples through a permission gate before invoking the header branch. Finally, at the **HEAVY** tier, a pre-integrated **XGBoost** model aggregates manifest tokens across all `classes*.dex` files into a 2,500-dimensional OR vector for gradient-boosted inference.

Where an original paper's architecture was not directly portable to ONNX or mobile inference, we adopted lightweight substitutes—replacing SVM/tree classifiers with small MLPs or linear heads—while preserving the published feature semantics. Each model is exported as a self-contained bundle containing `model.onnx`, an `export_manifest.json` describing input shapes and domains, threshold files, feature vocabulary or normalization assets, and a small set of parity samples. These bundles are copied directly into VigiDroid's asset directory, making the PC-to-phone handoff a straightforward file transfer.

3.1.1 On-Device System: VigiDroid

VigiDroid is an offline-first Android application that runs all inference locally using ONNX Runtime, without sending any APK data to a remote server. When a user initiates a scan, they first choose a scan depth—**Quick**, **Balanced**, or **Full**—which controls which model tiers are active. Quick mode runs only the LIGHT tier for a fast initial screen; Balanced adds the MID tier for better accuracy at moderate cost; Full enables all three tiers including the HEAVY XGBoost model.

For each APK found in the device's `Download/` folder, `ScanService` opens a metered session and runs the enabled models in sequence. Java feature extractors inside the app replicate the Python extraction logic from P2 exactly, using the vocabulary and normalization assets shipped in the export bundle—this replication is a strict requirement, since any drift between Python and Java feature semantics would silently invalidate on-device accuracy. After each model produces a score, `EnsemblePolicy` fuses the available outputs into a final verdict of `benign`, `malware`, or `uncertain`. Three fusion policies are supported: a legacy mode using only XGBoost and ByteCNN for backward compatibility; a default weighted-average mode that combines all models that ran, with weights derived from offline validation accuracy; and a tiered cascade mode that escalates from LIGHT to MID or HEAVY only when the lighter models are inconclusive.

Throughout each scan, VigiDroid records a detailed operational trace. Per-stage measurements include wall-clock time, whole-process CPU consumption, per-thread CPU time for the scan worker, native and Java heap deltas, proportional set size (PSS) changes, and battery percentage change over the session. These metrics are appended to a structured JSON log (`all_scan_metrics.json`) that accumulates across sessions and can be pulled from the device for post-hoc analysis.

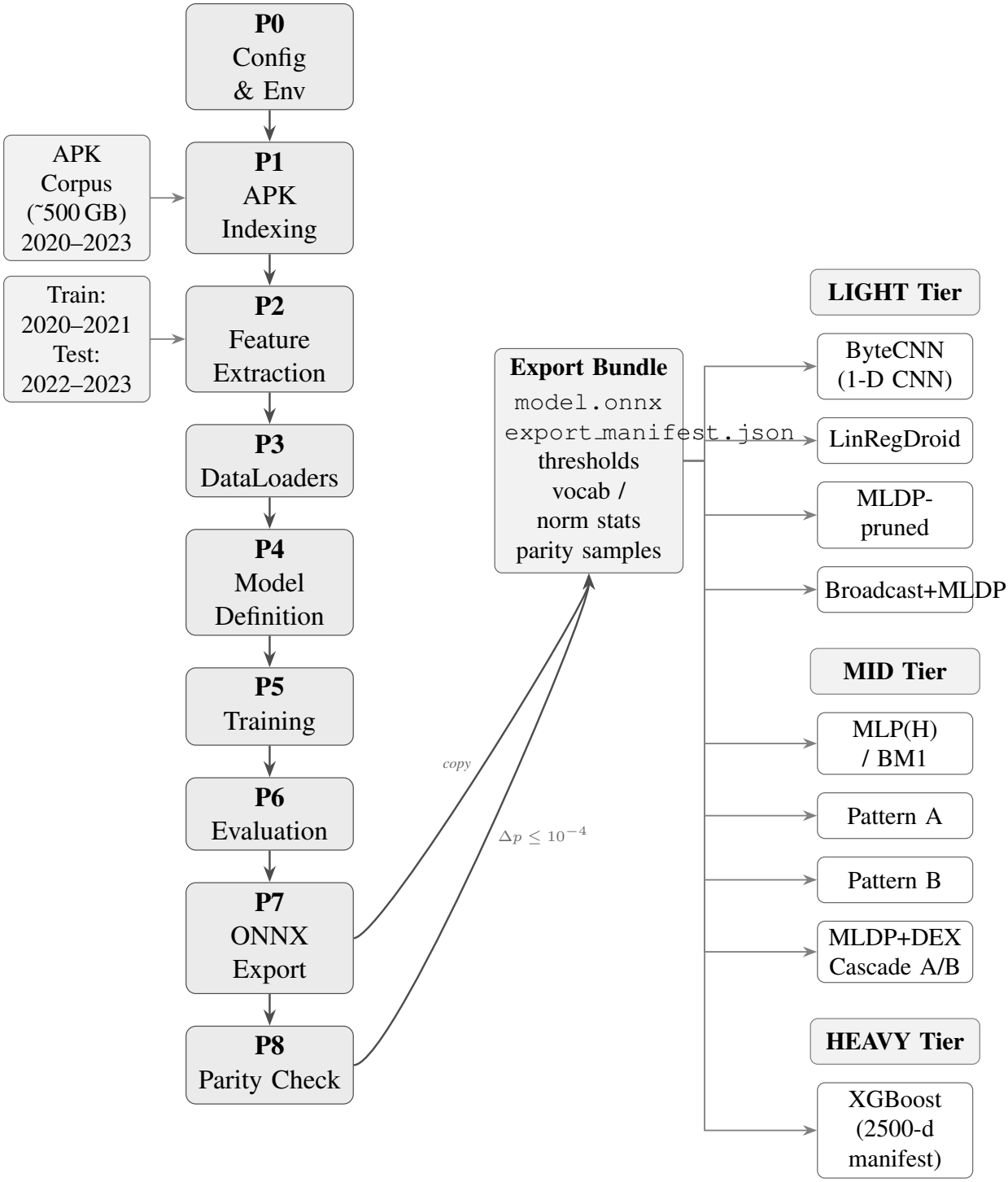

Figure 3.1: Offline VigiDroid model-development pipeline. The PC-side workflow indexes APKs, extracts features, trains and exports ONNX bundles, and organizes the exported detectors into LIGHT, MID, and HEAVY inference tiers.

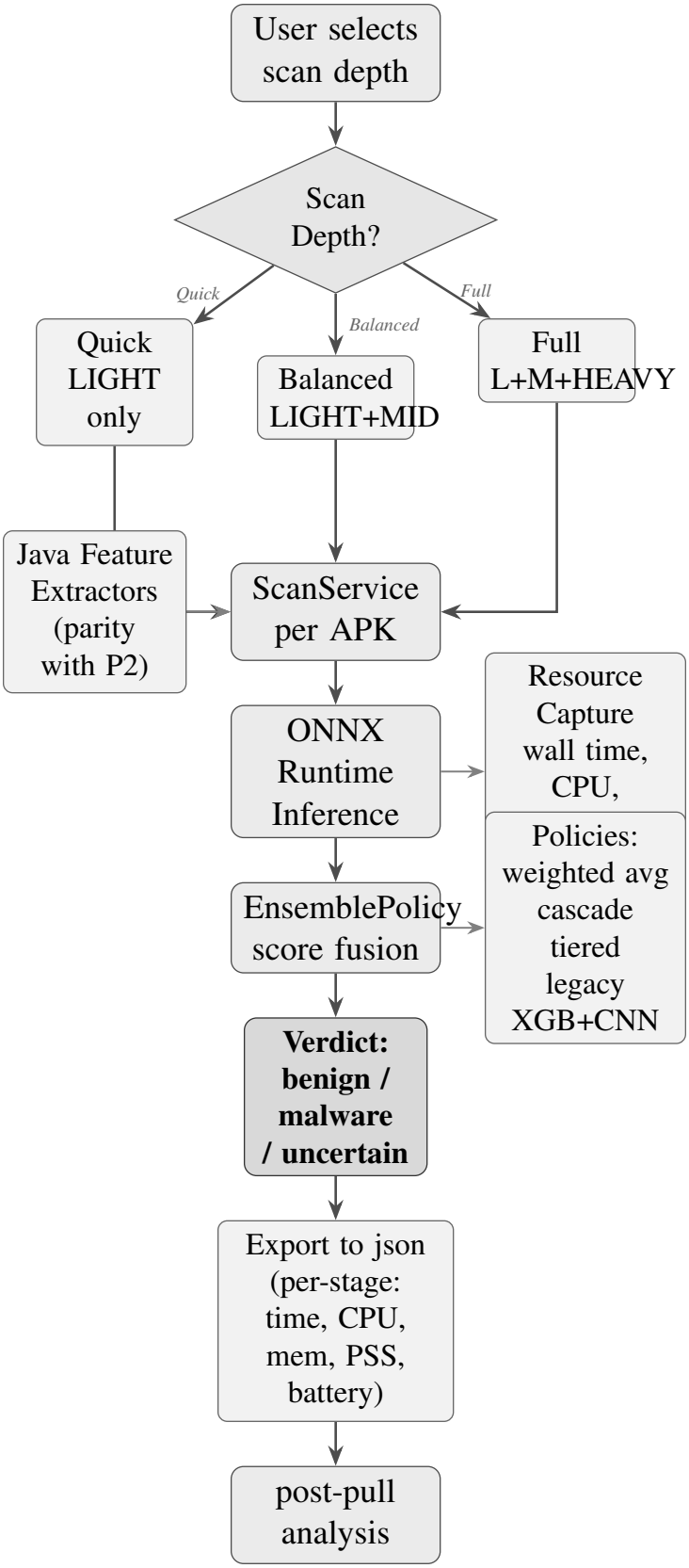

Figure 3.2: VigiDroid on-device application flow. The app selects a scan depth, runs the corresponding model tiers through local feature extraction and ONNX Runtime inference, fuses model scores, reports a verdict, and writes resource metrics for post-device analysis.

Chapter 4

Experiments

To validate the feasibility and performance of VigiDroid as a practical on-device malware detection system, a rigorous experimental pipeline was constructed. Unlike conventional static analysis frameworks that evaluate models solely in synthetic offline settings, the evaluation methodology of this research explicitly couples traditional classification metrics with live hardware-level metrology collected directly from physical Android devices.

This chapter details the data provenance, structural parsing configurations, model optimization procedures, and the design of the custom hardware profiling harness used to map the accuracy-resource frontier.

4.1 Dataset Provenance and Characterization

A foundational challenge in machine learning-based Android malware detection is ensuring resistance to temporal decay. Models trained on historical malware samples frequently suffer severe performance drops when faced with newer software mutations because of evolving API standards and novel obfuscation techniques. To address this, this study evaluates **VigiDroid** using a strict chronological data split rather than standard randomized K-fold cross-validation.

The experimental corpus was constructed from the public **Hugging Face 'android-apks' dataset**, comprising approximately 13,500 distinct Android applications spanned across the years 2020 through 2023. The dataset is bifurcated into two explicit operational splits to replicate real-world security deployment conditions:

- In-Sample Training and Development Split (2020–2021):Applications published during 2020 and 2021 serve as the foundational repository for feature discovery and model optimization. This subset is divided into a stratified configuration composed of a 70% training set, a 15% validation set, and a 15% development-testing set. All baseline training, hyper-

parameter grid-searches, and structural thresholds are calculated exclusively within this boundary.

- Out-of-Sample Temporal Holdout Split (2022–2023):Applications published during 2022 and 2023 are completely isolated during the model selection and training phases. This split functions as a strict temporal holdout to rigorously evaluate each model's capability to detect future zero-day threats and resist classification decay.

The categorical labels are uniformly structured as binary classifications: ‘0‘ representing verified benign applications and ‘1‘ representing verified malicious software strains.

—

4.2 Feature Engineering, Extraction, and Normalization

The extraction layer of *VigiDroid* is designed to ingest multi-source raw characteristics directly from compressed APK files without invoking resource-intensive terminal command disassemblers like ‘Apktool‘. The system maps out five distinct standalone feature sets and fusion combinations:

4.2.1 Manifest Permissions and Intents

Using the pythonic validation library ‘pyaxmlparser‘ for the background data pipelines, the framework targets the binary XML layout of the compressed ‘AndroidManifest.xml‘. Permissions (e.g., ‘android.permission.SEND_SMS‘) and intent filters are isolated and subjected to string normalization.

The raw strings are stripped of system prefixes and mapped to standardized, lower-case tokens (e.g., ‘send_sms‘) that match an internal, static vocabulary vocabulary built into the app assets. This transforms highly variable manifest declarations into fixed-width binary presence-absence feature vectors (1 for present, 0 for absent).

4.2.2 Statically Registered Broadcast Receivers

The framework scans for structural ‘¡receiver¿‘ blocks declared within the application's configuration blueprint. The specific system-wide intent action tags associated with these elements (such as ‘BOOT_COMPLETED‘ or ‘SMS_RECEIVED‘) are gathered, tokenized, and stored as dedicated binary flag components.

4.2.3 Header Metadata

Instead of decoding the complete compiled bytecode string graph from ‘classes.dex‘, *VigiDroid* implements a highly lightweight parsing routine that targets exclusively the fixed-size structural header bytes of the Dalvik Executable (DEX) format. This header exposes critical structural counts—such as the total number of method references, string indices, type IDs, and field references—providing deep insights into code complexity at a minimal processing cost.

4.2.4 Raw Tail Byte Sequences

For domain-agnostic sequence analysis, *VigiDroid* implements a specialized extraction layer that reads the exact last $N = 1024$ bytes (the tail) of the uncompressed raw APK archive. This tail sequence captures critical payload tables and directory offsets embedded within the ZIP archive architecture without requiring any extraction overhead.

—

4.3 Model Optimization, Compilation, and ONNX Export

The structural machine learning architectures selected for evaluation include a broad spectrum of modeling paradigms:

Linear/Statistical Baseline Model: A Multiple Linear Regression (MLR) wrapper and a calibrated Linear Support Vector Classifier (LinearSVC) are trained using Scikit-Learn. These models establish a baseline for pure, permission-driven manifest parsing.

Ensemble Tree Models:Extreme Gradient Boosting (XGBoost) models are trained to capture nonlinear feature interactions among manifest tokens and DEX header fields.

Deep Learning Models: Multilayer Perceptrons (MLPs) are developed for structural multi-source fusion, alongside an Adaptive Shrinkage Convolutional Neural Network (ASCNN) for multi-branch manifest and DEX data convergence. For raw byte processing, a 1-D Convolutional Neural Network (1-D CNN) is built to ingest the 1024 tail-byte array.

To enable execution within a native mobile context, all finalized model architectures are converted into the standardized **Open Neural Network Exchange (ONNX)** binary format. Models built using PyTorch or Scikit-Learn are run through optimization wrappers that map their operations to fixed-width, highly efficient ONNX primitives. These compiled ‘.onnx‘ files are bundled directly into the Android application’s asset directory, allowing them to be executed locally using the **ONNX Runtime (ORT)**mobile library.

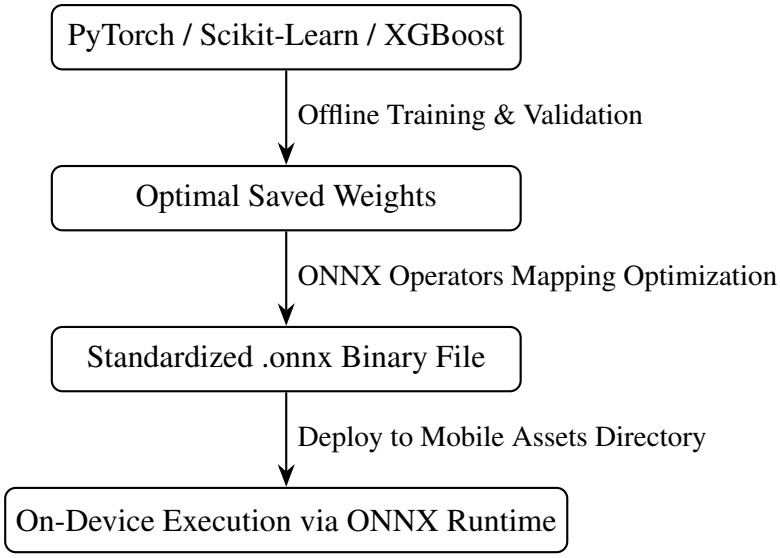

Figure 4.1: Model optimization and ONNX deployment pipeline

4.4 On-Device Hardware Profiling Infrastructure

The core contribution of our experimental setup is the design of an automated, native hardware-profiling infrastructure deployed directly on real physical test devices. This setup avoids using synthetic simulators, which frequently misrepresent true hardware overhead and thermal behavior.

The live benchmarking framework is built into a dedicated Android service (‘ScanService.onHandleWork‘) that acts as a secure processing container. When an target APK file is pointed out to the engine, the system launches an isolated background workflow managed by a sequence of specialized software components:

- ’FeatureContext‘ Allocation:Opens an optimized input stream to isolate file target spaces directly inside the local storage without exhausting system resources.
- ‘ScanOrchestrator‘ Gating: Executes the cascading engine. Depending on the experimental profile selected, it can isolate an individual model configuration or run the complete 4-tier cascading sequence.
- ‘ScanMetricsAssembler‘ Diagnostics: To isolate the exact resource footprint of feature extraction, tensor vectorization, and model inference, this component interfaces directly with low-level Android operating system diagnostics:
  - Execution Latency (CPU Time): Measured in milliseconds using precision system clocks (‘System.nanoTime()‘), tracking the exact execution duration of individual scanning stages.

  - Memory Footprint (RAM): Tracked by querying the Android Runtime memory allocation metrics and cross-referencing them with kernel memory stats (‘android.os.Debug.MemoryInf
revealing the exact RAM usage spikes caused by model loading and evaluation.
  - CPU Utilization: Profiled by tracking execution thread scheduling percentages through system diagnostics.

- ‘MetricsWriter‘ Encapsulation: The raw performance metrics and classification results are serialized into a standardized, local JSON-Lines archive (‘.jsonl‘) for downstream analysis. This file provides the underlying empirical evidence used to define the system’s accuracy-resource frontier.

Model	Features	Acc.	F1	ROC-AUC	CPU	Mem.	Batt.	Time	Feasible
XGBoost	Manifest perms, intents, APIs	0.9372	0.8884	0.9913	97.4 ms	0.02 MB	477.102	100.0 ms	low
1D-CNN	Last 1024 APK bytes	0.9005	0.8486	0.9592	1.7 ms	0.01 MB	8.433	1.7 ms	low
MLP-H	Dex header, sum-pooled	0.9686	0.9478	0.9675	0.7 ms	0.01 MB	1.997	0.4 ms	high
Dex+Manifest ASCNN	Dex header + manifest BoW	0.9411	0.9064	0.9685	7.8 ms	0.03 MB	36.045	7.9 ms	medium
Dex+Manifest Dual	Dex header + manifest, dual-branch	0.9686	0.9483	0.9805	7.9 ms	0.02 MB	36.84	7.9 ms	medium
LinRegDroid	Manifest permission flags	0.9175	0.8645	0.9259	1.0 ms	0.00 MB	3.008	0.7 ms	low
MLDP-pruned	MLDP-selected permissions	0.9319	0.8855	0.9017	0.6 ms	0.00 MB	1.61	0.4 ms	medium
Broadcast+MLDP	MLDP perms + receiver actions	0.9359	0.8928	0.9373	1.1 ms	0.00 MB	1.76	0.4 ms	medium
MLDP+Dex Cascade	MLDP perms + dex header	0.9581	0.9298	0.9469	0.8 ms	0.01 MB	4.919	0.4 ms	high
Dex+Broadcast Fusion	Dex header + receiver actions	0.9699	0.9503	0.9794	0.9 ms	0.01 MB	2.362	0.5 ms	high
Droidcat	Dynamic Analysis	0.9739	0.9783	0.98	–	–	–	–	–
SeqMobile	AndroidManifest.xml and DEX file	0.9767	0.9766	–	–	–	–	–	–

Table 4.1: Experimental comparison of on-device malware detection models. Offline metrics are computed on the computer temporal test split; resource columns are derived from on-device `all_scan_metrics.json`.

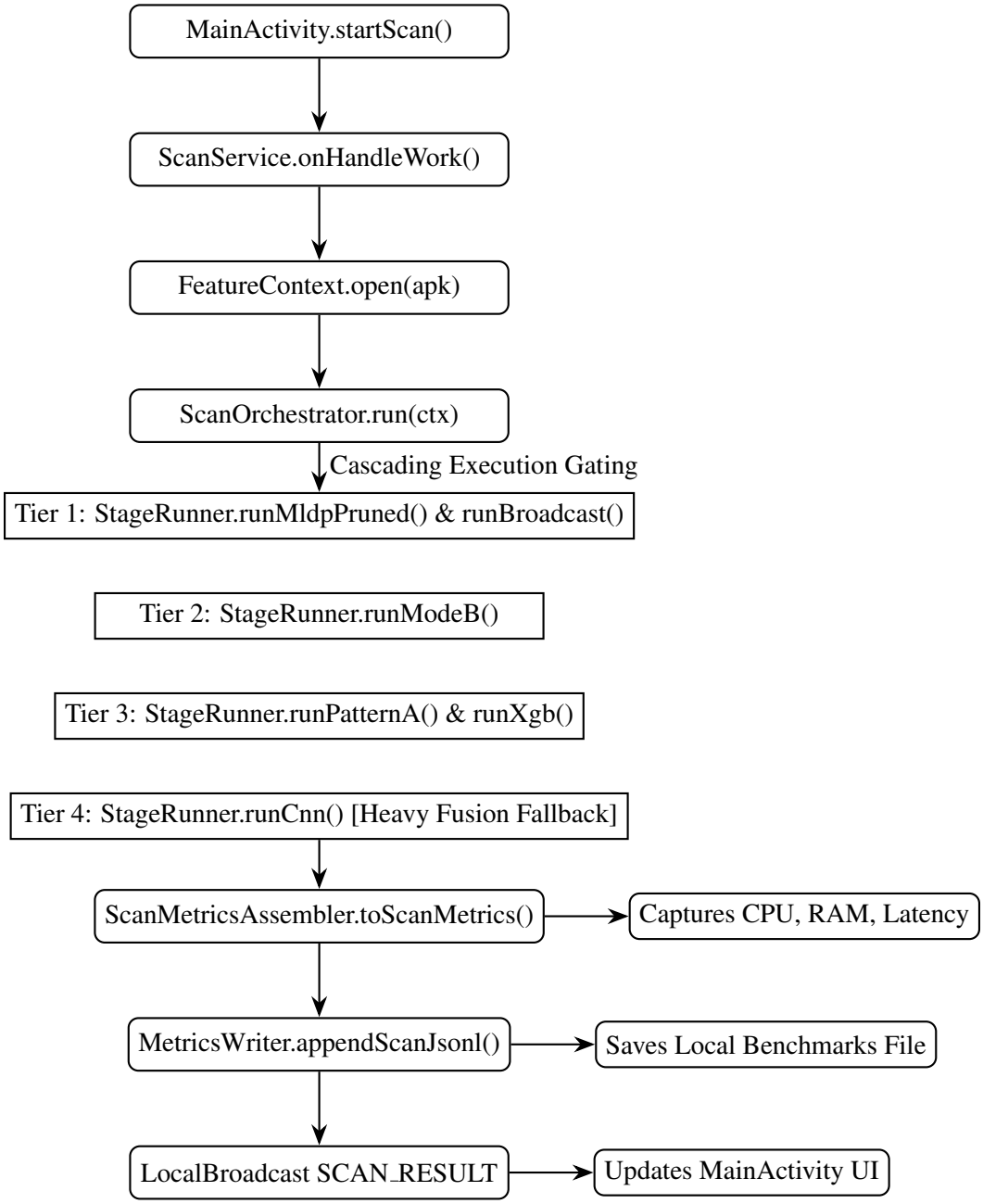

Figure 4.2: On-device hardware profiling workflow

Chapter 5

Professional Practices, Team Dynamics, and Lifelong Learning

5.1 Considerations of Professional Ethics, Responsibilities, and Norms of Engineering Practice

Engineering complex mobile security frameworks involves responsibilities that extend far beyond optimizing algorithmic accuracy. Developing an on-device malware detection system like VigiDroid requires balancing technical innovation with strict adherence to research ethics, software engineering norms, and societal data safety expectations.

5.1.1 Research and Data Ethics

The foundational ethical consideration of this study centers on data provenance and user privacy. To train and evaluate VigiDroid's multi-tier machine learning models, thousands of malicious and benign Android Package (APK) applications were compiled and indexed. Maintaining research integrity required ensuring that no real-world user data was harvested, intercepted, or compromised during both the offline dataset compilation and the live on-device testing phases. Furthermore, our benchmarks avoid attributing performance drops or false positives to specific commercial software or developers in a malicious light, preserving objective scientific neutrality. Academic honesty was strictly enforced through the meticulous citation of all foundational works—such as LinRegDroid, SeqMobile, and MSFDroid—ensuring clear boundaries between prior art and our original architectural contributions, such as the 4-tier cascading scan orchestration.

5.1.2 Practices and Conventional Norms

The system was engineered following rigid, industry-standard software development lifecycles (SDLC) and rigorous experimental design paradigms:

- Experimental Design Reproducibility: Data splits were organized using rigorous chronological, temporal holdouts (training on 2020–2021 data and validating against a 2022–2023 holdout set). This choice prevented data leakage and ensured the framework's evaluation mirrored a true zero-day production environment.
- Coding Standards and Version Control: The codebase was developed within a centralized monorepo (thesis_vigidroid), utilizing Git for version control. Code artifacts were modularized across distinct layers: a Java-backed native Android parsing engine for targeted AndroidManifest.xml binary XML decoding, and a separate pythonic pipeline for training, validating, and exporting models to standard ONNX formats.
- Testing and Profiling Metrology: Rather than relying purely on simulator benchmarks, we built a specialized hardware-profiling testing harness. Code blocks were subjected to regression testing to verify that streaming decompression did not leak file handles, and memory utilization was tracked locally using explicit device allocations to prevent execution faults or memory leaks during background service routines (ScanService.onHandleWork).

5.2 Reflection on Project Management, Team Dynamics, and Economic Decision Making

5.2.1 Project Management and Milestones

Executing a multifaceted research project involving machine learning, embedded optimization, and system-level Android engineering required structured project management. The lifecycle of VigiDroid was divided into iterative sprints, moving from initial literature review and feature exploration to native android compilation and final hardware profiling.
Progress was managed by tracking concrete engineering milestones, such as achieving a robust multi-source feature extraction pipeline, successfully exporting the calibrated XGBoost and 1-D CNN architectures to optimized ONNX binaries, and ultimately implementing the multi-tiered cascading execution logic.

5.2.2 Team Dynamics and Collaboration

Because this thesis was executed as a collaborative group study, maintaining healthy team dynamics and open communication channels was vital to delivering the project on time. Workloads were divided according to specialized core competencies:

Team Member Role	Primary Structural Responsibilities
Lead Infrastructure & Systems	Handled native Android integrations, stream-based APK zip parsing, and binary XML manifest extraction layers.
Machine Learning Pipeline Engineer	Managed feature tokenization dictionaries, model exploration, hyperparameter tuning, and ONNX conversion wrappers.
Hardware Profiling & Metrics Analyst	Built the on-device automated testing harness to capture CPU execution times, memory limits, and battery drain.

Table 5.1: Team member roles and primary structural responsibilities

To maintain momentum and alignment, the group established a rigid meeting cadence. Weekly coordination meetings were held to review active code changes, resolve API integration conflicts between the Python training scripts and the Android runtime environment, and debug classification anomalies. Furthermore, bi-weekly progress reviews with our thesis advisor ensured that our research directions remained tightly aligned with academic expectations and verified the validity of our hardware-profiling metrics.

5.2.3 Economic and Financial Decision-Making

A core objective of VigiDroid was to deliver an economically viable, highly efficient alternative to traditional commercial mobile security systems. Standard cloud-based mobile security architectures incur substantial operational costs, requiring continuous outlays for high-throughput cloud hosting, real-time API server maintenance, and substantial network bandwidth to offload user APK files for remote dynamic analysis.

VigiDroid completely eliminates these recurring infrastructure expenses by operating as a 100% offline, fully on-device application. By executing optimized, low-footprint models locally via ONNX Runtime, the system requires zero external cloud server infrastructure.

From an consumer-device standpoint, VigiDroid also makes highly conscious economic tradeoffs regarding hardware wear and tear. Running continuous, un-optimized dynamic analysis or heavy recurrent sequential loops directly on a smartphone rapidly degrades the device's lithium-ion battery health due to prolonged thermal stress and high CPU utilization. VigiDroid's 4-tier cascading scan orchestration acts as an economic safeguard for the user's physical hardware:

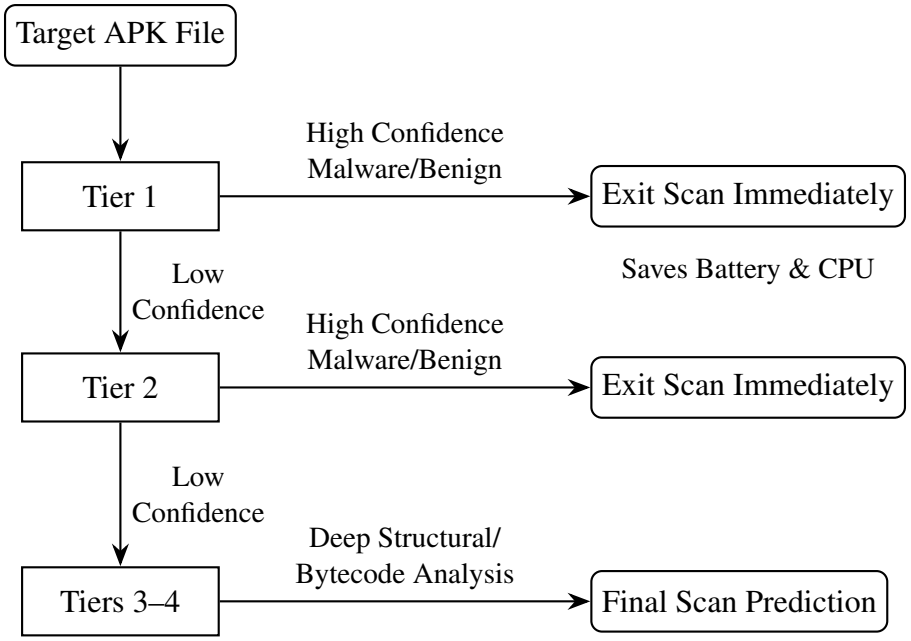

Figure 5.1: VigiDroid's cascading multi-tier scan orchestration

By terminating the scanning sequence immediately once a lightweight manifest tier yields a decisive prediction, the application drastically minimizes CPU cycles. This significantly reduces the device's battery consumption and limits thermal throttling, effectively extending the operational lifespan of the user's mobile hardware.

5.3 Reflection on Self-directed and Lifelong Learning to Navigate Technological Change

5.3.1 Self-Directed Learning Mechanics

The core architecture of VigiDroid required our team to bridge major knowledge gaps that extended well beyond the boundaries of our standard undergraduate computer science curriculum. While university coursework covered foundational concepts in machine learning and software engineering, implementing a fully functioning, on-device security system required extensive, independent self-directed learning:

- Binary Layout Parsing: We independently investigated the underlying structure of compiled Android packages, learning how to parse compressed Binary XML structures and extract raw structural metadata from Dalvik Executable (DEX) file headers without relying on heavy external third-party decompilation suites.
- On-Device Runtime Optimization: Deploying machine learning models directly onto

mobile hardware required mastering cross-platform compilation toolchains. We independently learned how to convert complex model structures into optimized, fixed-width ONNX binaries and integrate them into a native Android environment using ONNX Runtime.

- Hardware-Level Metrology: To accurately evaluate VigiDroid, we designed and built an automated hardware-profiling infrastructure from scratch. This involved exploring low-level Android operating system diagnostics, utilizing system-level benchmarking APIs, and capturing real-time hardware execution logs to measure exact CPU time and memory consumption.

5.3.2 Framework for Lifelong Learning

The mobile computing and cybersecurity fields are characterized by continuous, rapid evolution. As malware authors develop increasingly sophisticated obfuscation techniques, static signatures quickly become obsolete. VigiDroid is deliberately architected to embrace this constant technological shift.

The framework features a highly modular, decoupled design where the feature-extraction components are completely isolated from the downstream machine learning models. This ensures that the system's token vocabularies and underlying ONNX model files can be seamlessly updated over time without requiring a rewrite of the core native infrastructure.

Engaging with this thesis has provided our team with a robust methodology for lifelong learning. It has taught us how to systematically deconstruct unfamiliar technical ecosystems, adapt to rapidly changing software platform restrictions, and evaluate software performance through the pragmatic lens of real-world hardware constraints. This adaptability is an essential asset for navigating the future of engineering practice, enabling us to continuously master and deploy emerging technologies long after the conclusion of this study.

Chapter 6

Conclusion

6.1 Thesis Summary and Key Findings

This thesis has addressed a critical gap in mobile cybersecurity: the translation of theoretically highly accurate machine learning models into practical, resource-conscious, on-device detection mechanisms for the Android operating system. While contemporary academic literature heavily emphasizes deep learning topologies or continuous dynamic tracing, these paradigms remain heavily restricted by the rigid battery, CPU time, and operational sandboxing constraints of modern mobile hardware.

To overcome these roadblocks, we designed, implemented, and empirically validated **VigiDroid**, a fully offline, multi-tier cascading static analysis engine powered locally by ONNX Runtime. By parsing multi-source features directly from unextracted APK files, VigiDroid establishes a highly favorable accuracy-resource frontier.

The primary structural conclusions drawn from this research are as follows:

- The Potency of Structural Shortcuts: This study demonstrates that full decompilation or complex call-graph sequencing is unnecessary for effective triaging. Extracting structural metadata from the fixed-size ‘classes.dex‘ header requires sub-millisecond overhead while yielding deep structural insights into code complexity.
- The Optimality of Feature Fusion: Our experiments reveal that the **Dex+Broadcast Fusion** configuration represents an optimal engineering sweet spot. By combining shallow bytecode characteristics with statically registered broadcast receiver actions, this configuration achieves an accuracy of 96.99% and an F1-score of 0.9503, requiring an exceptionally low CPU inference time of just 0.9 ms.
- Validation of the Cascading Architecture: The implementation of an intelligent 4-tier cascading execution pipeline successfully solves the accuracy-resource dilemma. By routing

clear-cut benign or malicious samples out of the pipeline early using lightweight manifest and permission checks (Tiers 1 and 2), VigiDroid bypasses heavy raw byte 1-D CNN models for over 80% of test cases. This drastically reduces battery drain and limits thermal throttling across physical test devices.

- Resilience to Temporal Decay:** By evaluating VigiDroid against an out-of-sample temporal holdout split (2022–2023) completely distinct from the training history (2020–2021), we verified that structural metadata patterns remain remarkably resilient to software evolutionary shifts and zero-day variations over time.

6.2 Research Limitations

Despite its strong empirical performance, several inherent boundaries constrain the current implementation of VigiDroid:

- Susceptibility to Advanced Obfuscation:Because VigiDroid relies strictly on structural static characteristics, heavily obfuscated malware strains that utilize advanced runtime reflection, dynamic class loading (‘DexClassLoader‘), or native binary execution hooks via the Java Native Interface (JNI) can selectively mask their structural footprints within the DEX header and manifest.

- Fixed Vocabulary Bottlenecks: The on-device vectorization pipeline depends on precompiled, static vocabulary dictionaries for permissions, intents, and broadcast actions. If future iterations of the Android SDK introduce entirely new system-level permissions or unique, developer-defined custom intent patterns, the local vectorizer will label these tokens as out-of-vocabulary (OOV), potentially causing a minor reduction in long-term classification accuracy until a local dictionary update is deployed.

- Storage vs. Layer Complexity: Integrating deep learning components, such as multi-branch ASCNNs or 1-D CNN tail-byte layers, increases the final APK footprint due to the inclusion of fixed-width ONNX model binaries. Balancing the storage footprint of these binaries alongside local memory boundaries remains an ongoing trade-off.

6.3 Future Work Directions

The insights gained from developing VigiDroid open several promising avenues for future exploration:

- Quantization and Hardware Acceleration:Future iterations will explore 8-bit integer ($INT8$) quantization of the underlying deep learning weights prior to ONNX export. This step aims to shrink the model storage footprint by up to 75% while utilizing hardware-accelerated Neural Processing Units (NPUs) via the Android Neural Networks API (NNAPI) to push inference latencies below the microsecond threshold.

- Incremental, Privacy-Preserving On-Device Learning: To combat natural model accuracy decay without centralizing user app data, future work could investigate federated learning primitives. This would allow VigiDroid to update its downstream model weights locally on the device using user-verified feedback or security logs, securely sharing model updates across devices without violating privacy boundaries.

- Heuristic Manifest De-obfuscation: Integrating lightweight structural heuristic modules capable of identifying dead-code insertion or artificial manifest bloat (often appended by malware authors to alter raw byte distributions) will enhance the robustness of the initial triage tiers.

- Expansion to Native Code Analysis: Extending the metadata parsing engine to scan the structural headers of Executable and Linkable Format (‘.so‘) native libraries within the APK’s ‘lib/‘ directory will allow VigiDroid to detect native-level exploits without triggering the computational overhead of full reverse engineering.

As the Android ecosystem grows, mobile security must shift from passive cloud-dependent surveillance to active, edge-based defense. VigiDroid proves that local, resource-constrained static analysis is a viable first line of defense rather than a compromise. By intelligently structuring feature extraction and coordinating execution through a cascading pipeline, modern mobile devices can defend themselves autonomously—preserving user privacy, minimizing battery consumption, and securing the digital edge from emerging threats.

References

[1] D. Şahın, S. Akleylek, and E. Kiliç, "Linregdroid: Detection of android malware using multiple linear regression models-based classifiers," *IEEE Access*, vol. 10, pp. 14246–14259, 2022.

[2] F. Mohsen, H. Bisgin, Z. Scott, and K. Strait, "Detecting android malwares by mining statically registered broadcast receivers," in *2017 IEEE 3rd International Conference on Collaboration and Internet Computing (CIC)*, pp. 67–76, 2017.

[3] C. Hasegawa and H. Iyatomi, "One-dimensional convolutional neural networks for android malware detection," in *2018 IEEE 14th International Colloquium on Signal Processing Its Applications (CSPA)*, pp. 99–102, 2018.

[4] R. Feng, J. Q. Lim, S. Chen, S.-W. Lin, and Y. Liu, "Seqmobile: An efficient sequence-based malware detection system using rnn on mobile devices," in *2020 25th International Conference on Engineering of Complex Computer Systems (ICECCS)*, pp. 63–72, 2020.

[5] J. L. J. H. Z. Z. R. H. Tao Peng, Bochao Hu and X. Hu, "A lightweight multi-source fast android malware detection model," *Appl. Sci.*, 2022.

[6] S. Arshad, M. A. Shah, A. Wahid, A. Mehmood, H. Song, and H. Yu, "Samadroid: A novel 3-level hybrid malware detection model for android operating system," *IEEE Access*, vol. 6, pp. 4321–4339, 2018.

[7] M. Alzaylaee, S. Yerima, and S. Sezer, "Dl-droid: Deep learning based android malware detection using real devices," 11 2019.

Chapter 7

Algorithms

7.1 Sample Algorithm

In Algorithm 1 we show how to calcute $y = x^n$.

Algorithm 1 Calculate $y = x^n$

Require: $n \geq 0 \vee x \neq 0$
Ensure: $y = x^n$
  $y \leftarrow 1$
  if $n < 0$ **then**
    $X \leftarrow 1/x$
    $N \leftarrow -n$
  else
    $X \leftarrow x$
    $N \leftarrow n$
  end if
  while $N \neq 0$ **do**
    if N is even **then**
      $X \leftarrow X \times X$
      $N \leftarrow N/2$
    else $\{N$ is odd$\}$
      $y \leftarrow y \times X$
      $N \leftarrow N - 1$
    end if
  end while

Chapter 8

Codes

8.1 Sample Code

We use this code to find out...

```
#include <stdio.h>
int Fibonacci(int);

main()
{
  int n, i = 0, c;

  printf("Enter the value of n: ");
  scanf("%d",&n);

  printf("\nFibonacci series\n");

  for (c = 1 ; c <= n ; c++)
    {
      printf("%d\n", Fibonacci(i));
      i++;
    }

  return 0;
}

int Fibonacci(int n)
{
```

```
24   if (n == 0)
25     return 0;
26   else if (n == 1)
27     return 1;
28   else
29     return (Fibonacci(n-1) + Fibonacci(n-2));
30 }
```

8.2 Another Sample Code

```
1  SELECT associations2.object_id, associations2.term_id,
2         associations2.cat_ID, associations2.term_taxonomy_id
3  FROM (SELECT objects_tags.object_id, objects_tags.term_id,
4       wp_cb_tags2cats.cat_ID, categories.term_taxonomy_id
5  FROM (SELECT wp_term_relationships.object_id,
6       wp_term_taxonomy.term_id, wp_term_taxonomy.term_taxonomy_id
7  FROM wp_term_relationships
8  LEFT JOIN wp_term_taxonomy ON
9       wp_term_relationships.term_taxonomy_id =
10      wp_term_taxonomy.term_taxonomy_id
11 ORDER BY object_id ASC, term_id ASC)
12 AS objects_tags
13 LEFT JOIN wp_cb_tags2cats ON objects_tags.term_id =
14      wp_cb_tags2cats.tag_ID
15 LEFT JOIN (SELECT wp_term_relationships.object_id,
16      wp_term_taxonomy.term_id as cat_ID,
17      wp_term_taxonomy.term_taxonomy_id
18 FROM wp_term_relationships
19 LEFT JOIN wp_term_taxonomy ON
20      wp_term_relationships.term_taxonomy_id =
21      wp_term_taxonomy.term_taxonomy_id
22 WHERE wp_term_taxonomy.taxonomy = 'category'
23 GROUP BY object_id, cat_ID, term_taxonomy_id
24 ORDER BY object_id, cat_ID, term_taxonomy_id)
25 AS categories on wp_cb_tags2cats.cat_ID = categories.term_id
26 WHERE objects_tags.term_id = wp_cb_tags2cats.tag_ID
27 GROUP BY object_id, term_id, cat_ID, term_taxonomy_id
28 ORDER BY object_id ASC, term_id ASC, cat_ID ASC)
29 AS associations2
30 LEFT JOIN categories ON associations2.object_id =
```

```
31      categories.object_id
32 WHERE associations2.cat_ID <> categories.cat_ID
33 GROUP BY object_id, term_id, cat_ID, term_taxonomy_id
34 ORDER BY object_id, term_id, cat_ID, term_taxonomy_id
```

Generated using Undegraduate Thesis LaTeX Template, Version 2.2. Department of Computer Science and Engineering, Bangladesh University of Engineering and Technology, Dhaka, Bangladesh.

This thesis was generated on Thursday 11th June, 2026 at 10:50am.